

Chapter 6

Identifying Similarities in Data

IN THIS CHAPTER

Clustering data

Identifying hidden groups of similar information in your data

Finding associations among data items

Organizing data with biologically inspired clustering algorithms

There is so much data around us that it can feel overwhelming. Large amounts of information are constantly being generated, organized, analyzed, and stored. *Data clustering* is a process that can help you make sense of this flood of data by discovering hidden groupings of similar data items. Data clustering provides a description of your data that says, in essence, *your data contains x number of groups of similar data objects*.

Clustering — in the form of grouping similar things — is part of our daily activities. You use clustering any time you group similar items together. For example, when you store groceries in your fridge, you group the vegetables by themselves in the crisper, put frozen foods in their own section (the freezer), and so on. When you organize currency in your wallet, you arrange the bills by denomination — larger with larger, smaller with smaller. Clustering algorithms achieve this kind of order on a large scale for businesses or organizations — where datasets can comprise thousands or millions of data records associated with thousands of customers, suppliers, business partners, products, services, and so on.

In short, data clustering is an intelligent separation of data into groups of similar data items. The algorithms that do this task have been applied in fields such as biology, marketing, information retrieval, and the analysis of online social networks.

This chapter guides you through the mechanics of data clustering and outlines its importance in predictive analytics.

Explaining Data Clustering

A *dataset* (or data collection) is a set of items. For example, a set of documents is a dataset where the data items are documents. A set of social network users' information (name, age, list of friends, photos, and so on) is a dataset where the data items are profiles of social network users.

Data clustering is the task of dividing a dataset into subsets of similar items. Items can also be referred to as instances, observation, entities, data objects, or data items. In most cases, a dataset is represented in table format — *a data matrix*. A data matrix is a table of numbers, documents, or expressions, represented in rows and columns as follows:

- Each row corresponds to a given item in the dataset.

Rows are sometimes referred to as *items, objects, instances, or observations*.

- Each column represents a particular characteristic of an item.

Columns are referred to as *features or attributes*.

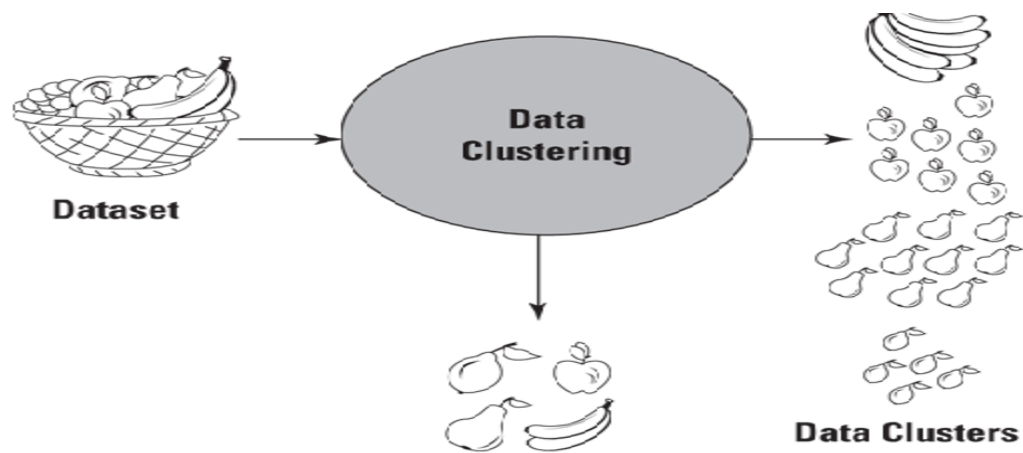
Applying data clustering to a dataset generates groups of similar data items. These groups are called *clusters* — collections of similar data items.

Similar items have a strong, measurable relationship among them — fresh vegetables, for example, are more similar to each other than they are to frozen vegetables or to fresh fruits — and clustering techniques use that relationship to group the items.

The strength of a relationship between two or more items can be quantified as a *similarity measure*: A mathematical function that computes the similarity between two data items. The results of that computation, called *similarity values*, essentially compare a particular data item to all other items in the dataset. Those other items will be either more similar or less similar in comparison to that specific item.

Computed similarities play a major role in assigning items to groups (*clusters*). Each group has an item that best represents it; this item is referred to as a *cluster representative*.

Consider a dataset that consists of several types of fruits in a basket, as shown in [Figure 6-1](#). The basket has fruits of different types such as apples, bananas, lemons, and pears. In this case, fruits are the data items. The data clustering algorithm extracts groups of similar fruits out of this dataset (basket of different fruits), assuming that the type of the fruit is unknown to the clustering algorithm.



odels

Chapter 6: Identifying Similarities in Data

FIGURE 6-1: Data clustering applied to a fruit dataset.

Imagine a robot or a technology that examines every fruit in the basket, and then automatically determines the weight and length for every fruit. Other possible features of fruits could include shape, size, color, and price among others. This step in the data analytics lifecycle is often referred as *feature extraction and feature selection*.

The robot doesn't know the names of fruits, and it will try to group similar fruits together by their features. Later in this process, a human being (such as a fruit expert) can label the discovered groups by looking at some of the data elements of each group. Or, if we leverage previous knowledge of some of the fruits in the basket, we can name the discovered groupings.

The results of the data clustering can be used to automatically assign new fruits to groups as they are added to the basket, and relatively predict their type just from knowing their features. This process in the data analytics lifecycle is often referred as data classification that will be covered in [Chapter 7](#).

The first step in a data clustering process is to translate this dataset into a data matrix: One way to model this dataset is to have the rows represent the items in the dataset (fruits); and the columns represent characteristics, or features, that describe the data items. There are algorithms that can be used specifically to extract and select features from raw data. [Chapter 9](#) covers the details.

In most cases, applying a data clustering technique to the fruit dataset as described above allows you to

- **Retrieve groups (clusters) of similar items.** You can tell that your fruit is of N number of groups. After that, if you pick a random fruit, you will be able to make a statement about that item as being part of one of the N groups.

- **Retrieve cluster representatives of each group.** In this example, a cluster representative would be picking one fruit type from the basket and putting it aside. The characteristics of this fruit are such that that fruit best represents the group of similar fruits it belongs to.

When you're done clustering, your dataset is organized and divided into natural groupings.

Motivation

Data clustering reveals structure in the data by extracting natural groupings from a dataset. Therefore, discovering clusters is an essential step toward formulating ideas and hypotheses about the structure of your data and deriving insights to better understand it.

Data clustering can also be a way to model data: It represents a larger body of data by clusters or cluster representatives.

In addition, your analysis may seek simply to partition the data into groups of similar items — as when *market segmentation* partitions target-market data into groups such as

- Consumers who share the same interests (such as Mediterranean cooking)
- Consumers who have common needs (for example, those with specific food allergies)

Identifying clusters of similar customers can help you develop a marketing strategy that addresses the needs of specific clusters.

Moreover, data clustering can also help you identify, learn, or predict the nature of new data items — especially how new data can be linked with making predictions. For example, in *pattern recognition*, analyzing patterns in the data (such as buying patterns in particular regions or age groups) can help you develop predictive analytics — in this case, predicting the nature of future data items that can fit well with established patterns.

The fruit basket example uses data clustering to distinguish between different data items. Suppose your business assembles custom fruit baskets, and a new exotic fruit is introduced to the market. You want to learn or predict which cluster the new item will belong to if you add it to the fruit basket. Because you've already applied data clustering to the fruit dataset, you have four clusters — which makes it easier to predict which cluster (specific type of fruit) is appropriate for the new item. All you have to do is to relatively compare the unknown fruit's features (such as weight, length, size, price, and shape) to the other four clusters' representatives and identify which cluster is the best match. Although this process may seem obvious for a person working with a small dataset, it isn't so obvious at a larger scale especially when you have large number of features. The complexity becomes exponential when the dataset is large, diverse, and relatively incoherent — which is why clustering algorithms exist: Computers do that type of work best.

A similar, common, and practical example is the application of data clustering to e-mail messages, dividing a dataset of e-mails into spam and non-spam clusters. An ideal spam-detection tool would need to first divide a dataset (all past e-mails) into two types (groups): spam

and non-spam. Then the tool predicts whether unknown incoming e-mail is spam or non-spam. This second step, often referred to as data classification, is covered in detail in [Chapter 7](#).

The goal would be to minimize the number of spam e-mails that end up in your inbox and also minimize the number of legitimate e-mails that end up in your spam folder. (As you've no doubt noticed, this kind of clustering and classification isn't quite perfected yet.)

Data clustering is used in several fields:

- **Computer imaging:** Data clustering is part of *image segmentation* — the process of dividing a digital image into multiple segments in order to analyze the image more easily. Applying data clustering to an image generates segments (clusters) of contours that represent objects — aiding the detection of threatening health conditions in medical diagnosis and in the screening of airport baggage for suspicious materials.
- **Information retrieval:** Here the aim is to search and retrieve information from a collection of data, for example a set of documents. Dividing a collection of documents into groups of similar documents is an essential task in information retrieval. Data clustering of documents by topic improves information search.
- **Medicine:** Applying data clustering to a matrix of gene expression (often known as *microarray gene expression data*) from different cancer-diagnosed patients can generate clusters of positively and negatively diagnosed cancer patients, which can help predict the nature of new cases.
- **Marketing:** Using data clustering to group customers according to similar purchase behavior improves the efficiency of targeted marketing.
- **Law enforcement:** In social network analysis, the same data-clustering techniques that can help detect communities of common interests can also help identify online groups involved in suspicious activity.

Converting Raw Data into a Matrix

Before you can extract groups of similar data items from your dataset, you might need to represent your data in a tabular format known as a *data matrix*. This is a preprocessing step that comes before data clustering.

Creating a matrix of terms in documents

Suppose the dataset that you're about to analyze is contained in a set of Microsoft Word documents. The first thing you need to do is to convert the set of documents into a data matrix. Several commercial and open-source tools can handle that task, producing a matrix (often known as a *document-term matrix*), in which each row corresponds to a document in the dataset. Examples of these tools include RapidMiner, and R text-mining packages such as *tm* that can convert unstructured text data into a form that the clustering algorithm can use to discover clusters.

The next section explains one possible way to how documents can be converted into a data matrix.

A *document* is, in essence, a sequence of words. A *term* is a set of one or multiple words.

Every term that a document contains is mentioned either once or several times in the same document. The number of times a term is mentioned in a document can be represented by *term frequency* (TF), a numerical value.

We construct the matrix of terms in the document as follows:

- The terms that appear in all documents are listed across the top row.
- Document titles are listed down the leftmost column.
- The numbers that appear inside the matrix cells correspond to each term's frequency.

For example, in [Table 6-1](#), Document A is represented as a set of numbers (5,16,0,19,0,0.) where 5 corresponds to the number of times the term *predictive analytics* is repeated, 16 corresponds to the number of times *computer science* is repeated, and so on. This is the simplest way to convert a set of documents into a matrix.

TABLE 6-1 Converting a Collection of Documents into a Matrix

	<i>Predictive Analytics</i>	<i>Computer Science</i>	<i>Learning</i>	<i>Clustering</i>	<i>2013</i>	<i>Anthropology</i>
Document A	5	16	0	19	0	0
Document B	8	6	2	3	0	0
Document C	0	5	2	3	3	9
Document D	1	9	13	4	6	7
Document E	2	16	16	0	2	13
Document F	13	0	19	16	4	2

Term selection

One challenge in clustering text documents is determining how to select the best terms to represent all documents in the collection. How important a term is in a collection of documents

can be calculated in different ways. If, for example, you count the number of times a term is repeated in a document and compare that total with how often it recurs in the whole collection, you get a sense of the term's importance relative to other terms.

Basing the relative importance of a term on its frequency in a collection is often known as *weighting*. The weight you assign can be based on multiple principles, of which two common examples are

- Terms that appear several times in a document are favored over terms that appear only once.
- Terms that are used in relatively few documents are favored over terms that are mentioned in all documents.

If (for example) the term *century* is mentioned in all documents in your dataset, then you might not consider assigning it enough weight to have a column of its own in the matrix. There are many weighting techniques that are widely used in practical text mining, such as *term frequency-inverse document frequency* (TF-IDF). TF-IDF tries to measure the importance of a word to both a document and a collection of documents. Similarly, if you're dealing with a dataset of users of an online social network, you can easily convert that dataset into a matrix. User IDs or names will occupy the rows; the columns will list features that best describe those users. (An example of a data matrix of social network users appears later in this chapter.)

Identifying Groups in Your Data

An *algorithm* is a step-by-step process used for solving a problem. One of the most popular and simple algorithm used in data clustering, *K-means*, is named after the algorithm's input and output: **K** is an input to the algorithm; it stands for the number of groupings that the algorithm must extract from a dataset, expressed algebraically as K-means, are outputs of the algorithm; they refer to the set of representatives that represent *k* clusters. A *cluster representative* is the statistical mean of all the data items in the same cluster. The next section explains in detail how to derive a cluster representative (that is, a mean).

K-means clustering algorithm

A K-means algorithm divides a given dataset into *k* clusters. The algorithm performs the following operations:

1. Pick *k* random items from the dataset and label them as initial cluster representatives. In order to improve the performance of the clustering algorithm, it's preferable that the *k* random items you pick as initial representatives that are well spaced in distance.
2. Associate each remaining item in the dataset with the nearest cluster representative, using a distance measure (such as Euclidean distance, explained later in this chapter).
3. Recalculate the new clusters' representatives.
4. Repeat Steps 2 and 3 until the clusters don't change.

A representative of a cluster is the mathematical *mean* (average) of all items that belong to the same cluster. This representative is also called a *cluster centroid*.

[Table 6-2](#) shows some features of three fruits (data points) from the fruits' dataset. We will assume that the items in [Table 6-2](#) are assigned to the same cluster so we show you how you can calculate a cluster representative of items belonging to the same cluster. In this example we only consider the length and weight as features of these similar fruits for simplicity.

[TABLE 6-2](#) Three Items from the Fruit Dataset

<i>Item</i>	<i>Feature#1 Weight (Ounces)</i>	<i>Feature#2 Length (Inches)</i>
1	6.5	6
2	5.5	7
3	6	6.5

[Table 6-3](#) shows the calculations of a cluster representative of three items (for example, bananas) that belong to the same cluster. The cluster representative is a vector of two attributes. Its attributes are the average of the attributes of the items in the cluster in question.

[TABLE 6-3](#) Calculating the Representative of Three Items

<i>Item</i>	<i>Feature#1 Weight (Ounces)</i>	<i>Feature#2 Length (Inches)</i>
1	6.5	6
2	5.5	7
3	6	6.5
Cluster Representative (Centroid Vector)	$(6.5+5.5+6)/3=6$	$(6+7+6.5)/3=6.5$

The dataset shown in [Table 6-4](#) consists of seven customers' ratings of two products, A and B. The ranking represents the number of points (between 0 and 20) that each customer has given to a product — the more points given, the higher the product is ranked. Using a K-means algorithm and assuming that k is equal to 2, the dataset will be partitioned into two groups.

The rest of the procedure looks like this:

1. **Pick two random items (preferably far apart from each other in distance) from the dataset and label them as cluster representatives (as shown in [Table 6-5](#)).**

[Table 6-6](#) shows the initial step of selecting random centroids from which the K-means clustering process begins. The initial centroids are selected randomly from the data that you're about to analyze. In this case, you're looking for two clusters, so two data items are randomly selected: Customers 1 and 5. At first, the clustering process builds two clusters around those two initial (randomly selected) cluster representatives. Then the cluster representatives are recalculated; the calculation is based on the items in each cluster.

2. **Inspect every other item (customer) and assign it to the cluster representative to which it is most similar.**

Use the *Euclidean distance* to calculate how similar an item is to a group of items:

Similarity of Item I to Cluster X =

The values x_i are the numerical values of the features that describe the item in question. The values μ_j are the features (mean values) of the cluster representative (centroid), assuming that each item has n features.

For example, consider the item called Customer 2 (3, 4): The customer's rating for Product A was 3 and the rating for Product B was 4. The cluster representative feature, as shown in [Table 6-6](#), is (2, 2). The similarity of Customer 2 to Cluster 1 is calculated as follows:

Similarity of Item 2 to Cluster 1 =

Here's what the same process looks like with Cluster 2:

Comparing these results, you assign Item 2 (that is, Customer 2) to Cluster 1 because the numbers say Item 2 is more similar to Cluster 1.

3. **Apply the same similarity analysis to every other item in the dataset.**

Every time a new member joins a cluster, you must recalculate the cluster representative, following the example shown in [Table 6-3](#).

The use of K-means is an iterative process. At each iteration, the algorithm steps shown here are repeated until the clusters show no further changes. [Tables 6-6](#) and [6-7](#) show these iterations and assignments of items to clusters. At the end of Iteration 1, Cluster 1 contains Items 1, 2, and 3; Cluster 2 contains Items 4, 5, 6 and 7. The new cluster representatives are (3.67, 4.67) for Cluster 1 and (8.25, 10.75) for Cluster 2.

[Table 6-6](#) depicts the results of the first iteration of K-mean algorithm. Notice that k equals 2, so you're looking for two clusters, which divides a set of customers into two meaningful groups. Every customer is analyzed separately and is assigned to one of the clusters on the basis of the customer's similarity to each of the current cluster representatives. Note that cluster representatives are updated each time a new member joins a cluster.

4. **Iterate the dataset again, going through every element; compute the similarity between each element and its current cluster representative.**

Notice that Customer 3 has moved from Cluster 1 to Cluster 2. This is because Customer 3's distance to the cluster representative of Cluster 2 is closer than to the cluster representative of Cluster 1.

[TABLE 6-4](#) Dataset of Customer Ratings for Products A and B

<i>Customer ID</i>	<i>Customer Ratings of Product A</i>	<i>Customer Ratings of Product B</i>
1	2	2
2	3	4
3	6	8
4	7	10
5	10	14
6	9	10
7	7	9

[TABLE 6-5](#) Selecting Random Initial Cluster Representatives

<i>Cluster Representative Centroid Vector)</i>	
Cluster 1	Customer ID#1 (2, 2)
Cluster 2	Customer ID#5 (10,14)

[TABLE 6-6](#) First Iteration of Applying a K-Means Algorithm to the Dataset of Customer Ratings

<i>Iteration#1</i>	<i>Customer Cluster 1</i>		<i>Customer Cluster 2</i>	
<i>Customer to be examined</i>	<i>Customer IDs belonging to Cluster 1</i>	<i>Cluster Representative</i>	<i>Customer IDs belonging to Cluster 2</i>	<i>Cluster Representative</i>
	1	(2, 2)	5	(10, 14)
2	1, 2	(2.5, 3)	5	(10, 14)
3	1, 2, 3	(3.67, 4.67)	5	(10, 14)
4	1, 2, 3	(3.67, 4.67)	4, 5	(8.5, 12)
6	1, 2, 3	(3.67, 4.67)	4, 5, 6	(8.67, 11.33)
7	1, 2, 3	(3.67, 4.67)	4, 5, 6, 7	(8.25, 10.75)

TABLE 6-7 Second Iteration of Applying a K-Means Algorithm to the Dataset of Customer Ratings

<i>Iteration#2</i>	<i>Customer Cluster 1</i>		<i>Customer Cluster 2</i>	
<i>Customer to be examined</i>	<i>Customer IDs belonging to Cluster 1</i>	<i>Cluster Representative</i>	<i>Customer IDs belonging to Cluster 2</i>	<i>Cluster Representative</i>
1	1, 2, 3	(3.67, 4.67)	4, 5, 6, 7	(8.25, 10.75)
2	1, 2, 3	(3.67, 4.67)	4, 5, 6, 7	(8.25, 10.75)
3	1, 2	(2.5, 3)	3, 4, 5, 6, 7	(7.8, 10.2)
4	1, 2	(2.5, 3)	3, 4, 5, 6, 7	(7.8, 10.2)
5	1, 2	(2.5,3)	3, 4, 5, 6, 7	(7.8, 10.2)

<i>Iteration#2</i>	<i>Customer Cluster 1</i>		<i>Customer Cluster 2</i>	
<i>Customer to be examined</i>	<i>Customer IDs belonging to Cluster 1</i>	<i>Cluster Representative</i>	<i>Customer IDs belonging to Cluster 2</i>	<i>Cluster Representative</i>
6	1, 2	(2.5, 3)	3, 4, 5, 6, 7	(7.8, 10.2)
7	1, 2	(2.5, 3)	3, 4, 5, 6, 7	(7.8, 10.2)

[Table 6-7](#) shows a second iteration of K-means algorithm on customer data. Each customer is being re-analyzed. Customer 2 is being assigned to Cluster 1 because Customer 2 is closer to the representative of Cluster 1 than Cluster 2. The same scenario applies to Customer 3. Notice that a cluster representative is being recalculated (as in [Table 6-3](#)) each time a new member is assigned to a cluster.

Clustering by nearest neighbors

Nearest Neighbors is a simple algorithm widely used to cluster data by assigning an item to a cluster by determining what other items are most similar to it. A typical use of the Nearest Neighbors algorithm follows these general steps:

1. Derive a similarity matrix from the items in the dataset.

This matrix, referred to as the *distance matrix*, will hold the similarity values for each and every item in the data set. (These values are elaborated in detail in the next example.)

2. With the matrix in place, compare each item in the dataset to every other item and compute the similarity value (as shown in the preceding section).

Initially, every item is assigned to a cluster of one item.

3. Using the distance matrix, examine every item to see whether the distance to its neighbors is less than a value that you have defined.

This value is called the *threshold*.

4. The algorithm puts each element in a separate cluster, analyzes the items, and decides which items are similar, and adds similar items to the same cluster.
5. The algorithm stops when all items have been examined.

Consider, for example, a dataset of eight geographical locations where individuals live. The purpose is to divide these individuals into groups based on their geographical locations, as determined by the Global Positioning System (see [Table 6-8](#)).

[TABLE 6-8](#) GPS Dataset

<i>Individual ID</i>	<i>GPS Longitude</i>	<i>GPS Latitude</i>
1	2	10
2	2	5
3	8	4
4	5	8
5	7	5
6	6	4
7	1	2
8	4	9

[Table 6-8](#) shows a simple dataset of individuals' geographic data. For purposes of simplicity, global positional longitude and latitude are depicted as whole numbers. Assume that all the data collected about these eight individuals was collected at a specific point in time.

As with K-means, the first pre-step is to calculate the similarity values for every pair of individuals. One way to calculate a similarity between two items is to determine the Euclidean distance (as described in the preceding section). The similarity value between two points is calculated as shown earlier in [Table 6-9](#).

[TABLE 6-9](#) Determining the Degree of Similarity (Euclidean Distance) Between Individuals

<i>Indivi dual#</i>	<i>Indivi dual#</i>	<i>Indivi dual#</i>	<i>Indivi dual#</i>	<i>Indivi dual#</i>	<i>Indivi dual#</i>	<i>Indivi dual#</i>	<i>Indivi dual#</i>
1	2	3	4	5	6	7	8

	<i>Indivi dual# 1</i>	<i>Indivi dual# 2</i>	<i>Indivi dual# 3</i>	<i>Indivi dual# 4</i>	<i>Indivi dual# 5</i>	<i>Indivi dual# 6</i>	<i>Indivi dual# 7</i>	<i>Indivi dual# 8</i>
Indivi dual# 1	0	5	8.49	3.61	7.07	7.21	8.06	2.24
Indivi dual# 2		0	6.08	4.24	5	4.12	3.16	4.47
Indivi dual# 3			0	5	1.41	2	7.28	6.40
Indivi dual# 4				0	3.61	4.12	7.21	1.41
Indivi dual# 5					0	1.41	6.71	5
Indivi dual# 6						0	5.39	5.39
Indivi dual# 7							0	7.62
Indivi dual# 8								0

Similarity between Item A and Item B =

Here $f_{a,1}$ is the first feature of Item A, $f_{a,2}$ is the second feature of Item A, and corresponding values labeled b represent the features of Item B. The variable n is the number of features. In this example, n is 2. For example, the similarity between Item 1 and Item 2 is calculated as follows:

Similarity between Item 1 and Item 2 =

On the basis of this measurement of similarity between items, you can use the Nearest Neighbor algorithm to extract clusters from the dataset of geographical locations.

The first step is to place the individual whose ID is 1, longitude is 2, and latitude is 10 in cluster C1. Then go through all remaining individuals computing how similar each one is to the individual in C1. If the similarity between Individual 1 and another Individual x is less than 4.5 (the user-provided threshold value beyond which an item is too dissimilar — the user of the algorithm can set or choose this value), then Individual x will join C1; otherwise you create a new cluster to accommodate Individual x .

[Table 6-9](#) shows the similarities and numerical relationships between Individuals 1 through 8. The similarity of these data elements is calculated as a Euclidean distance (explained earlier in this chapter); for example, the similarity between Individual 1 and Individual 5 is 7.07. Individuals with similarity values closer to 0 have greater similarity. Half the matrix isn't filled because the matrix is *symmetric* (that is, the similarity between Individuals 1 and 4 is the same as the similarity between Individuals 4 and 1, a property guaranteed by any appropriate similarity measure).

You have now assigned Individual 1 to the first cluster (C1). The similarity between Individual 1 and Individual 2 is equal to 5, which is greater than the threshold value 4.5. A new cluster is generated — and Individual 2 belongs to it. At this stage, you have two clusters of one item each: $C1 = \{\text{Individual 1}\}$ and $C2 = \{\text{Individual 2}\}$.

Moving the focus to Individual 3, you find that the similarity between Individual 3 and Individual 2 & 1 is larger than the threshold value 4.5. Thus you assign Individual 3 to a new cluster containing one item: $C3 = \{\text{Individual 3}\}$.

Moving to Individual 4, you calculate how similar Individual 4 is to Individuals 1, 2, and 3. The nearest (most similar) to Individual 4 happens to be Individual 1. The similarity between 4 and 1 is approximately 3.61, which is less than the threshold value 4.5. Individual 4 joins Individual 1 in Cluster C1. The clusters constructed so far are: $C1 = \{\text{Individual 1, Individual 4}\}$, $C2 = \{\text{Individual 2}\}$ and $C3 = \{\text{Individual 3}\}$.

Next is to examine Individual 5 and calculate how similar it is to Individuals 1, 2, 3, and 4. The item nearest in distance (most similar) to Individual 5 is Individual 3. The similarity is which is less than the threshold value of 4.5. Thus Individual 5 joins C3. The clusters constructed so far are: $C1 = \{\text{Individual 1, Individual 4}\}$, $C2 = \{\text{Individual 2}\}$ and $C3 = \{\text{Individual 3, Individual 5}\}$.

When you examine Individual 6 and calculate how similar it is to Individuals 1, 2, 3, 4, and 5, you discover that Individual 5 is nearest (most similar) to Individual 6. Thus Individual 6 joins C3. The clusters constructed so far are: $C1 = \{\text{Individual 1, Individual 4}\}$, $C2 = \{\text{Individual 2}\}$ and $C3 = \{\text{Individual 3, Individual 5, Individual 6}\}$.

When you examine Individual 7 and calculate how similar it is to Individuals 1, 2, 3, 4, 5, and 6, you find that the nearest (most similar) item to Individual 7 is Individual 2. Thus Individual 7 joins C2. The clusters constructed so far are: $C1 = \{\text{Individual 1, Individual 4}\}$, $C2 = \{\text{Individual 2, Individual 7}\}$ and $C3 = \{\text{Individual 3, Individual 5, Individual 6}\}$.

When you examine Individual 8, and calculate its similarity to Individuals 1, 2, 3, 4, 5, 6, and 7, you find that the nearest (most similar) item to Individual 8 is Individual 4. Thus Individual 8 joins C1.

The groups of similar data items constructed so far, containing items most similar to each other, are

- $C1 = \{\text{Individual 1, Individual 4, Individual 8}\}$
- $C2 = \{\text{Individual 2, Individual 7}\}$
- $C3 = \{\text{Individual 3, Individual 5, Individual 6}\}$

Density-based algorithms

Density-based clustering algorithms are a set of machine learning techniques that can discover regions where a number of data elements are relatively close in distance to each other. These areas of data elements create *dense regions of data points*. A density-based algorithm discovers dense areas of data points and starts on building clusters around those dense areas.

One of the most popular density-based algorithms is *density-based spatial clustering of applications with noise* (DBSCAN).

DBSCAN is inspired by the human way of discovering dense areas. In fact, human beings usually detect crowds following dense regions of people. The notion of density in data clustering is based on the fact that data groupings can be discovered by following dense regions.

In order to understand how DBSCAN works, it's important to note that DBSCAN algorithm requires two predefined parameters:

- ϵ denotes the radius of a neighborhood.

A neighborhood of a given point x is the circular region of radius ϵ that contain data points that are in a distance less than ϵ .

- MinPts denotes Minimum points to be considered for a cluster.

DBSCAN is an iterative algorithm; at every iteration it tries to discover the following three types of data points:

- **Core points:** A point that has more than a minimum points (MinPts) within a radius ϵ . Those points are said to be *directly reachable* from core point X.
- **Border points:** A point that has fewer points than MinPts points within an ϵ radius, but is in the neighborhood of a core point. A neighborhood of given point X is the set points that are within ϵ distance from X.
- **Noise points:** Points that aren't either core points or border points.

In summary, DBSCAN algorithm finds points in the densest regions, known as core points (analogous to cluster representative in K-means). Core points can later be used to generate clusters. Unlike K-means, DBSCAN can explicitly discover noise points.

The main steps of the iterative algorithm can be summarized as follows:

1. Find and label data points as core, border and noise.
2. Report and eliminate noise data points
3. For every core point c that hasn't been assigned to a cluster, create a new cluster with the point C and all the points that are density connected to P.

Consider the following terminologies around the concept of density-connected points:

1. Two points X and Y are density connected if there is a point Z such that both X and Y are density reachable from Z.
 2. We say that a point X is density-reachable from a point Y if there is exist a set of points $Q_1, \dots, Q_{n-1}$, where Q_{i+1} is directly density-reachable from Q_i
- Assign border points to the cluster of the closest core point.

It is important to note the following items about DBSCAN algorithm:

- DBSCAN can handle noise points.
- DBSCAN doesn't require the number of clusters an input to the algorithm.
- DBSCAN offers a sense of the density of the data.
- DBSCAN can handle clusters of different shapes and sizes in most cases.
- DBSCAN can find to some extent arbitrarily shaped clusters.
- DBSCAN is very sensitive to the input parameters radius ϵ and MinPts.
- DBSCAN is very sensitive to the input parameters radius ϵ and MinPts.

Finding Associations in Data Items

The use of predictive analytics as a data-mining tool also seeks to discover hidden relationships among items in your data. These hidden relationships are called *mining association rules*.

Consider a large dataset of customer transactions, where a *customer transaction* consists of the product(s) purchased by a customer at a given time. In a scenario like this one, the purpose of predictive analytics *as a tool* is to identify associations between products in the dataset. An association between two products is a *relation*, which can help the analyst discern a pattern and derive a rule from the raw data of customer transactions. An example of such a rule could be grocery-buying patterns: If a customer purchases butter and bread, he or she is also likely to buy milk. The rule discovered in this case can be written as

$\{\text{butter, bread}\} \rightarrow \{\text{milk}\}.$

In data-mining terms, $\{\text{butter, bread}\}$ is called a *basket*. A real-world basket contains items, of course, and so does this basket: butter and bread. The discovered rule just described is that if a basket contains the items butter and bread, then it's also very likely to contain milk.

Finding such *association rules* in a dataset of customer transactions helps a company (in this case, a grocery store) maximize revenue by deciding which products should be on sale, how to position products in the store's aisles, and how and when to offer promotional pricing.

Analyzing the data generated by past transactions in order to maximize profit is a common practice. Sales data collected regularly (daily, weekly, monthly) from point-of-sale systems such as online stores, supermarkets, bookstores, and restaurants is referred to as *basket data* — which is, in this case, essentially large-scale data about sales transactions. Association rules are generated with a score known as *confidence* — which refers to how likely they are to hold true. For example, if a generated rule shows that 98% of the people who purchased butter and bread also purchased milk, that percentage value (98%) is the *confidence value*.

Other terms associated with a rule are *antecedent* (the “if” part of an “if-then” statement) and the *consequent* (the “then” part of the “if-then”). In the preceding example, the antecedent is butter and bread; milk is the consequent.

In practice, your company will use predictive analytics to retrieve association rules from a customer database. The analyst issues queries whose purpose is to find rules that are either related to the antecedent (what was bought) or rules that can lead to the consequent (what can be expected to be bought).

In another example, consider a coffee shop manager who wants to maximize profit using association rules as a data-mining tool. The store manager would request items like these:

- Generate all rules that have *croissant* in the antecedent and *café latte* in the consequent.

Such rules would help the manager develop recommendations for which products to sell together with croissants; if café latte is prominent as a consequent, it's highly likely that the recommendation will be to sell café latte with croissants.

- Generate all rules that have *chocolate chip cookie* as an antecedent.

These rules may help outline and design a plan for increasing sales of chocolate chip cookies.

- Generate all rules that have *espresso* as an antecedent.

These rules would determine the products whose sales may be affected if the store runs out of espresso.

Apriori algorithm

One of the most popular data mining algorithms for discovering association rules is the *Apriori algorithm*. The algorithm relies on knowledge of frequent itemset properties. It's an iterative algorithm that starts by determining the set of frequent 1-itemset (L1), then it determines the set of frequent two-itemset L2 using L1, and so on. The algorithm is available in major predictive analytics tools, such as R, Apache Mahout, and Weka. For an implementation of Apriori in R, see <https://cran.r-project.org/web/packages/arules/arules.pdf>. For an implementation of Apriori in Weka, view <https://arxiv.org/ftp/arxiv/papers/1406/1406.7371.pdf>

In a recent project, use cases were identified and a roadmap developed for a large financial organization where large-scale predictive analytics frameworks can be leveraged to streamline business operations. One of the subject areas was predictive maintenance (the process of capturing and analyzing system logs as indicators to predict failure in a system). Subsequently, those indicators can be used to develop decisions to act ahead of time to prevent system interruption.

In this case, the Apriori algorithm was applied in the field of predictive maintenance. The purpose of the project was simply to predict software failure before it happens.

Here's how Apriori was used to mine associations that could be used for predictive maintenance.

The first step was a data exploration step to identify the most critical hardware and software failures that the organization had over the past few years:

- A data dictionary for external and internal data sources was built to help mine frequent items set of event that led to failure.
- Monitoring tools that are used within the organization for hardware and/or software logs were identified.

Some of the major common software failures can be summarized as

- Process unable to be started.
- Processes are started in an incorrect sequence which leads to incorrect process status.
- Performance degradation/crash due to implementation failure (such as memory leak or incorrect procedure call).
- Poorly engineered software.
- Absence of automated patch management systems.
- License(s) expiration.
- Distributed denial-of-service attack (DDOS).

If a service or network is incapable of handling traffic, it may bottleneck the services or machines and make all services on the machine or network inaccessible.

Some of the common hardware failures can be summarized as

- **Hard drives:** File system corruption, mechanical failure, moisture, and heat.
- **CPUs:** Heat, over-utilization, overclocking, and surges causing damage to transistors.
- **Power supplies:** Overloading and overheating (fan failure, dust, dirt, and surges).
- **Network ports:** Damage.

The second step was to identify data sources, such as

- **Event logs:** Automated event-logging mechanisms or manual logs.
- **System Active Reports (SAR):** Reports from API of operating systems.
- **Self-Monitoring Analysis and Reporting Technology (SMART):** Disk drive health capture of syslog data, hardware agent logging, memory, CPU and memory, fan RPM, and power consumption.
- **Software logs** (such as Tableau, SharePoint, and SQL servers).

The third step was to assemble a team of data scientists, a data architects, business users that will be using the framework, a data analytics vendor for Hadoop, and business users who will use the framework. ([Chapter 16](#) has details about Hadoop.)

The data architects designed an ETL process (extract, transform and load) and moved data from data sources into a data hub under Hadoop Distributed File System (HDFS).

After the data was stored into HDFS, it was pre-processed. MapReduce jobs were written to perform these tasks:

- Convert logs into one common format.

Syslog, event logs, hardware sensor logging, and environmental logs are semi-structured (for example, per data time: *List<time>*, *List(events>*).

- Convert unstructured data from web crawling to capture common hardware, software name, failure type, and severity of the failure.

After the data was pre-processed and harmonized into one data matrix, many possible predictive models for predicting failures were considered. One of the algorithms that was adopted was Apriori. Apriori was able to mine events from the data hub that were correlated. For example, some of the results were in the following format:

- {Event A of type Error, Event B of type Error} → {Event C of type Critical}
- {Event D of type Critical, Event E of type Error} → {Event F of type System Outage}

From those association rules, several paths to failure were detected that could be used as predictors for impending failure, based on historical events.

Applying Biologically Inspired Clustering Techniques

Nature is a collection of beautiful, simple, and efficient systems. Even a natural process as simple as an apple falling from a tree inspired Newton to identify the law of gravity. But nature is also the best place to look for patterns that suggest solutions to complex problems — natural collective behaviors such as schools of fish, flocking birds, and marching ants have led experts in predictive analytics to design biologically inspired data-clustering algorithms.

Designing a biologically inspired solution involves these steps.

1. Observe and analyze a phenomenon in nature.
2. Mimic or model the behavior and design a representation for it that a computer can comprehend.
3. Use the representation to solve a real-world problem.

In data clustering, two widely-used algorithms purely inspired by nature are based on the flocking of birds and the collective behavior of ant colonies. Both algorithms can model and cluster your data in a simple and natural way.

Birds flocking: Flock by Leader algorithm

Imagine birds' flocking behavior as a model for your company's data. Each data item corresponds to a single bird in the flock; an appropriate visual application can show the flock in action in an imaginary visual space. Your dataset corresponds to the flock. The natural flocking behavior corresponds to data patterns that might otherwise go undiscovered. The aim is to detect swarms (data clusters) among the flocking birds (data elements).

Flocking behavior has been used in real-life applications such as robotics-based rescue operations and computer animation. For example, the producer of the movie *Batman Returns* generated mathematical flocking behavior to simulate bat swarms and penguin flocks.

The use of flocking behavior as a predictive analytics technique — analyzing a company's data as flocks of similar data elements — is based on the dynamics behind flocking behavior as it appears in nature.

Flocking behavior of birds, fish, flies, bees, and ants is a self-organizing system; the individuals tend to move in accordance with both their environment and neighboring individuals.

In a flock of birds, each bird applies three main rules while flocking:

- **Separation** keeps a bird apart from its nearest flock mates.
- **Alignment** allows a bird to move along the same average heading as that of its flock mates.
- **Cohesion** keeps the bird within the local flock.

Each bird in a flock moves according to these rules. A bird's *flock mates* are birds found within a certain distance of the bird, and a certain distance from each other. To avoid collision between birds, a minimum distance must be kept; it can also be mathematically defined. Such are the rules that orchestrate flocking behavior; using them to analyze data is a natural next step.

Consider a dataset of online social network users. Data clustering can identify social communities that share the same interests. Identifying social communities in a social network is a valuable tool that can transform how organizations think, act, operate, and manage their marketing strategies.

Suppose that Zach is an active social network user whose profile has over a thousand friends. If you know who Zach's top ten closest online friends are, who among them he interacts with the most, *and* (from other data analytics sources) that Zach has just bought a book named *Predictive Analytics For Dummies*, you can then target his closest friends (those who are most similar to Zach) and suggest the same book to them, rather than trying to suggest it to all 1,000 of his friends.

In addition, detection of online social clusters can be extremely valuable for intelligence agencies, especially if (say) Zach and some of his closest friends — but not all of his friends — are involved in suspicious activities.

How do you obtain a dataset of social network users? Well, some of the data and tools are already available: Major social networks and micro-blog websites such as Facebook and Twitter provide an application programming interface (API) that allows you to develop programs that can obtain public data posted by users. Those APIs offered by Twitter are referred to as Twitter Streaming APIs. They come in three main types: public, user, and site streams:

- **Public streams** allow a user to collect public tweets about a specific topic or user, or support an analytics purpose.
- **User streams** allow a user to collect tweets that are accessible by the user's account.
- **Site streams** are for large-scale servers that connect to Twitter on behalf of many users.

Now, suppose you use such a program to download users' data and organize it into a tabular format such as the matrix shown in the next figure. [Table 6-10](#) shows a simple matrix that records the online interactions of Zach's online friends over two different weeks. This dataset consists of seven elements and seven features. The features as shown in the table column are the number of interactions between each member and the other members.

[TABLE 6-10](#) Week 1 of a Dataset of a Social Network User's Weekly Interactions

Social Member Network	Interactions with John	Interactions with Mike	Interactions with Zach	Interactions with Emma	Interactions with Kellie	Interactions with Nicole	Interactions with Arthur
John	-	10	10	12	4	4	10

<i>Social Member Network</i>	<i>Interactions with John</i>	<i>Interactions with Mike</i>	<i>Interactions with Zach</i>	<i>Interactions with Emma</i>	<i>Interactions with Kellie</i>	<i>Interactions with Nicole</i>	<i>Interactions with Arthur</i>
Mike		-	5	5	56	57	5
Zach			-	6	41	4	4
Emma				-	28	8	8
Kellie					-	5	5
Nicole						-	4
Arthur							-

For example, the number of online interactions between Mike and John over Week 1 is 10. These interactions include — but aren't limited to — chat conversations, appearances in the same photos, tweets, and e-mail exchanges. After building this dataset, the flocking behavior will be applied to detect communities (clusters) and visualize this data as a flock of data items.

Let us walk you through how a computer applies bird-flocking behavior to detect communities of common interests.

Take an empty piece of paper and imagine you have information about 20 users in a social network that you want to analyze. On that piece of paper, draw 20 dots randomly distributed all around. Each dot represents a user in the social network. If you move dots as if they were birds, they will move according to two central principles:

- The similarity between those social users in real life
- The rules that produce flocking behavior as you see it in nature

Let's say you have information about online interactions of those 20 members over a period of 10 days. You can gather information about those daily users on a daily basis, including data about their interactions with one another.

For example, Mike and John appeared in the same photo album, they attended the same event, or they “liked” the same page in an online social network. This information will be processed, analyzed, and converted into a matrix of numbers, with these results:

- The dots (birds) in representing Mike and John move toward each other (flocking attraction rule).
- The dots move with the same speed (flocking alignment and cohesion rules) on that piece of paper (flocking virtual space).
- In one day, several real-world interactions will drive the movements of those dots (birds) in your paper.
- Applying flocking behavior over 20 days will naturally lead to forming swarms of dots (birds) that are similar in some way.

There are many ways to apply the bird-flocking behavior to discover clusters in large datasets. One of the most recent variations is the Flock by Leader machine-learning clustering algorithm, inspired by the discovery of bird leaders in the pigeon species. The algorithm predicts data elements that could potentially lead another group of data objects. A leader is assigned, and then the leader initiates and leads the flocking behavior. Over the course of the algorithm, leaders can become followers or outliers. In essence, this algorithm works in a way that follows the rules of “survival of the fittest.”

The *Flock by Leader* algorithm was first introduced by Prof. Abdelghani Bellaachia and Prof. Anasse Bari in “Flock by Leader: A Novel Machine Learning Biologically Inspired Clustering Algorithm,” published as a chapter in the proceedings of the Advances in Swarm Intelligence at Volume 7332 of the series Lecture Notes in Computer Science.

[Tables 6-10](#) and [6-11](#) show one possible way to represent data generated by online social exchanges over two weeks. [Table 6-10](#) shows that Zach interacted 41 times with Kellie and four times with Arthur.

[TABLE 6-11](#) Week 2 of a Dataset of a Social Network User’s Weekly Interactions

Social Member Network	Interactions with John	Interactions with Mike	Interactions with Zach	Interactions with Emma	Interactions with Kellie	Interactions with Nicole	Interactions with Arthur
John	-	10	12	10	0	10	8
Mike		-	50	2	0	0	5
Zach			-	9	0	1	3

<i>Social Member Network</i>	<i>Interactions with John</i>	<i>Interactions with Mike</i>	<i>Interactions with Zach</i>	<i>Interactions with Emma</i>	<i>Interactions with Kellie</i>	<i>Interactions with Nicole</i>	<i>Interactions with Arthur</i>
Emma				-	2	2	1
Kellie					-	4	9
Nicole						-	1
Arthur							-

[Figure 6-2](#) outlines how to apply the bird-flocking algorithm to analyze social network data. As depicted in the figure, each member is represented by a bird in virtual space. Notice that

- The birds are initially dispersed randomly in the virtual space.
- Each bird has a velocity and a position associated with it.
- Velocity and position are calculated for each bird, using three vectors: separation, attraction, and alignment.
- Each bird moves according to the three vectors, and this movement produces the flocking behavior seen in nature.

FIGURE 6-2: Discovering communities (clusters) by representing social network users as flocking birds in a virtual space.

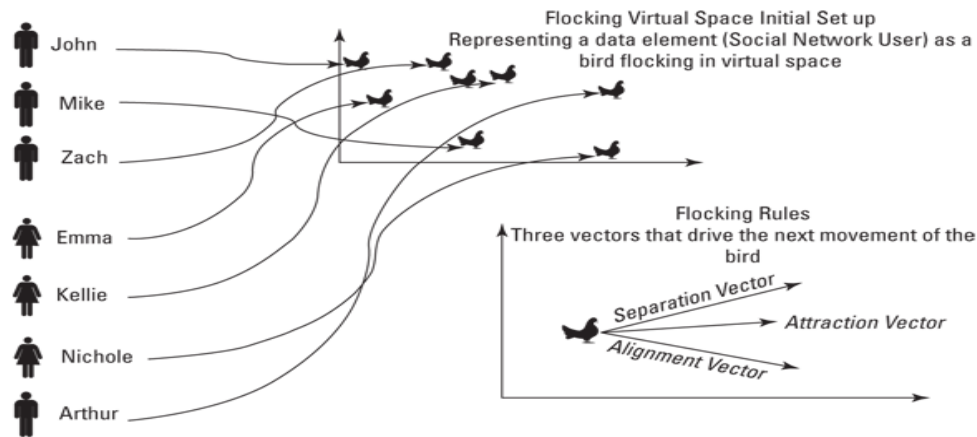


FIGURE 6-2: Discovering communities (clusters) by representing social network users as flocking birds in a virtual space.

[Figure 6-3](#) illustrates the dynamics of applying flocking behavior to the initial birds shown in [Figure 6-2](#).

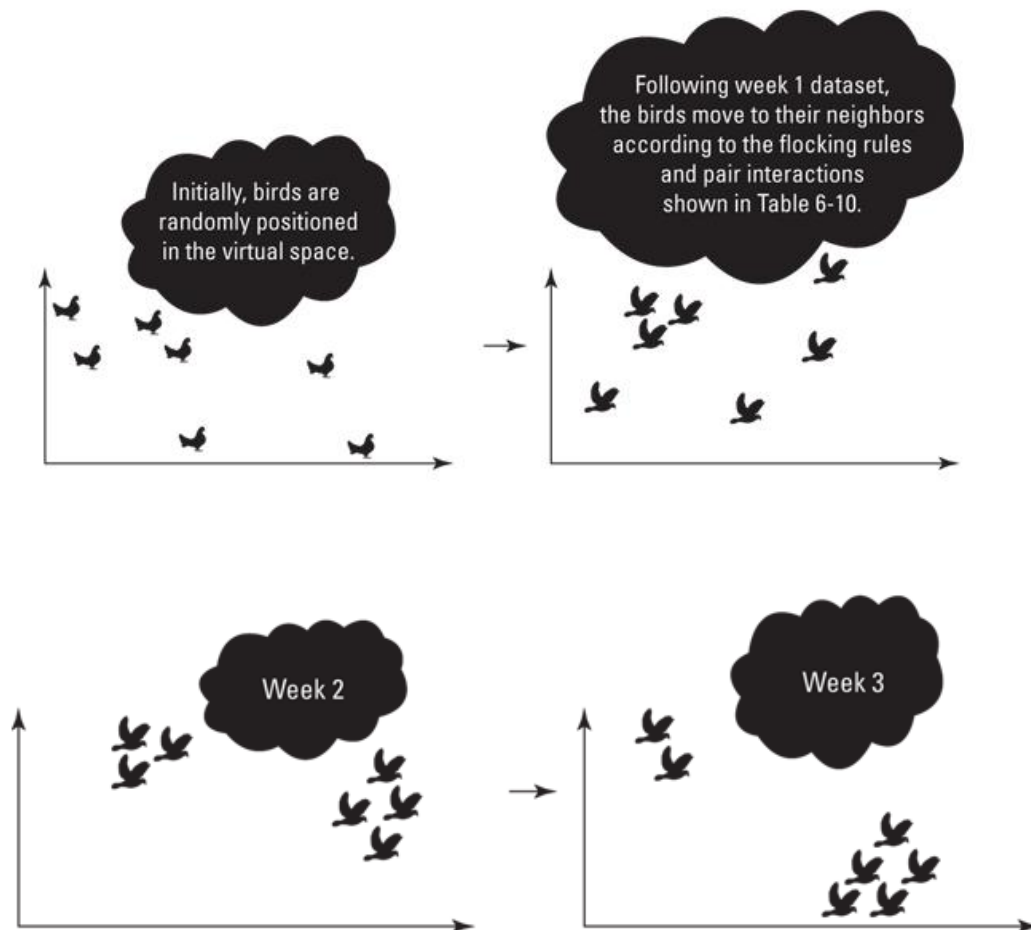


FIGURE 6-3: Applying flocking behavior to detect social communities based on weekly interactions.

FIGURE 6-3: Applying flocking behavior to detect social communities based on weekly interactions.

Here interaction data is analyzed weekly to find similar social networks' users. Each week the birds can be visualized in a simple grid. The positions of these birds reflect the interactions of actual individuals in the real world.

Ant colonies

Another natural example of self-organizing group behavior is a colony of ants hunting for food. The ants collectively optimize their track so that it always takes the shortest route possible to a food target. Even if you try to disturb a marching colony of ants and prevent them from getting to the food target — say, by blocking their track with a finger — they get back on track quickly and (again) find the shortest way possible to the food target, all of them avoiding the same obstacles while looking for food. This uniformity of behavior is possible because every ant deposits a trail of pheromones on the ground. Okay, how is that related to predictive analytics? Surprisingly closely. Read on.

Consider an army of ants idle in their nest. When they start looking for food, they have absolutely no information about where to find it. They march randomly until an individual ant finds food; now the lucky ant (call it Ant X) has to communicate its find to the rest of the ants — and to do that, it must find its way back to the nest.

Fortunately, Ant X was producing its own pheromones the whole time it was looking for food; it can follow its own trail of pheromones back to the nest. On its way back to the nest, following its own pheromone trail, Ant X puts more pheromones on the same trail. As a result, the scent on Ant X's trail will be the strongest among all the other ants' trails. The strongest trail of pheromones will attract all the other ants that are still searching for food. They'll stop and follow the strongest scent. As more ants join Ant X's trail, they add more pheromones to it; the scent becomes stronger. Pretty soon, all the other ants have a strong scent to follow.

So far, so good: The first ant that discovers food leads the rest of the ants to it by reinforcing scent. But how do the ants discover the shortest path? Well, the trail with the strongest pheromone will attract the most ants — but when large numbers of ants are involved, the shortest trails that reach the food will allow more trips than the longer trails. If several ants have discovered the same source of food, the ants that took the shortest path will do more trips in comparison to ants that follow longer paths — hence more pheromones will be produced on the shortest path. The relationship between individual and collective behavior is an enlightening natural example.

In [Figure 6-4](#), every dot represents a document. Assume that the black dots are documents about predictive analytics and the white dots are documents about anthropology. Dots representing the different types of documents are randomly distributed in the grid of five cells. “Ants” are deployed randomly in the grid to search for similar documents. Every cell with a value in it represents an instance of a “pheromone.” Using the document matrix, each cell's “pheromone” value is calculated from the corresponding document.

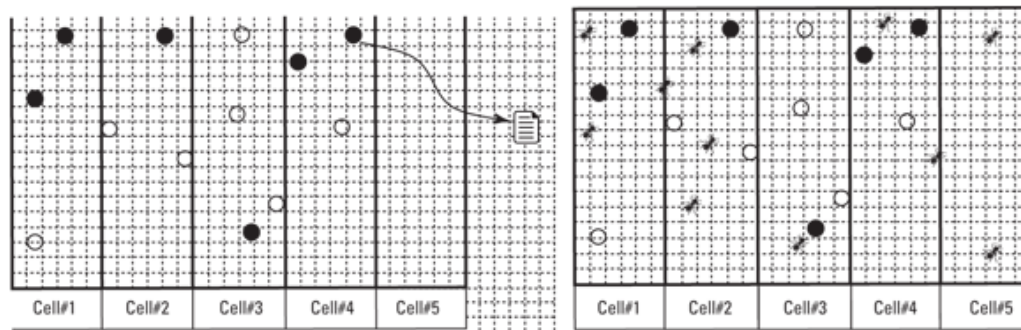


FIGURE 6-4: Setting up the initial phase of clustering documents, using a model based on ant-colony behavior.

FIGURE 6-4: Setting up the initial phase of clustering documents, using a model based on ant-colony behavior.

Okay, how does an ant colony's collective intelligence produce a model for effectively clustering data? The answer lies in a simple analogy: Ants are searching for food in their environment, much as we're searching for clusters in a dataset — looking for similar documents within a large set of documents.

Consider a dataset of documents that you want to organize by topic. Similar documents will be grouped in the same cluster. Here's where the ant colony can provide hints on how to group similar documents.

Imagine a two-dimensional (2D) grid where we can represent documents as dots (often referred to as *vectors* with coordinates). The 2D grid is divided into cells. Each cell has a "pheromone" (value) associated with it. (Later in this section, we show how to calculate this value.) Briefly, the "pheromone" value distinguishes each document in a given cell.

The dots are initially distributed randomly — and every dot in the grid represents a unique document. The next step is to deploy other dots randomly on the 2D grid, simulating the ant colony's search for food in its environment. Those dots are initially scattered in the same 2D grid with the documents. Each new dot added to the grid represents an ant. Those "ants," often referred to in the ant-colony algorithm as *agents*, are moving in the 2D grid. Each "ant" will either pick up or drop off the other dots (documents), depending on where the documents best belong. In this analogy, the "food" takes the form of documents sufficiently similar that they can be clustered.

In [Figure 6-5](#), an "ant" walks randomly in the grid; if it encounters a document, it can perform one of two actions: pick or drop. Each cell has a "pheromone intensity" (a relative size of numerical value) that indicates how similar the document is to the other documents (dots) residing near the document in question — the one an "ant" is about to either pick up or drop. Note that the "ant" in Cell 3 will pick up the black-dotted document because the white "pheromone" value is dominating; and move to a cell where the value is close (similar) to what's in Cell 4 (several black dots). The search keeps iterating until the clusters form.

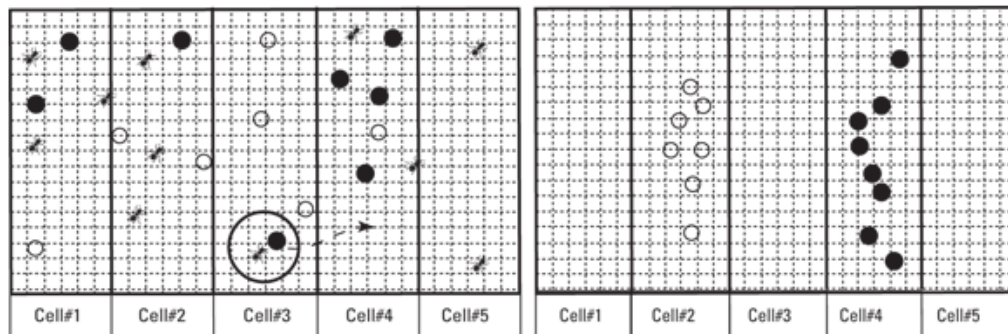


FIGURE 6-5: Deploying “ants” into the documents’ virtual space to discover similar documents.

FIGURE 6-5: Deploying “ants” into the documents’ virtual space to discover similar documents.

In effect, the “ant” moves documents from one cell to another to form clusters by performing either one of only two actions: picking up a document or dropping a document. When the “ants” started moving randomly on the grid, encountering a dot (document) results in the “ant” picking up a document from its current cell, moving with it, and dropping it into a cell in which it had sufficient similarity to fit.

How would an “ant” determine the best cell in which to drop a document? The answer is that the values in the cells act like “pheromones” — and every cell in the 2D grid contains a numerical value that can be calculated in a way that represents a document in the cell. Remember that each document is represented as a set of numbers or a vector of numerical values. The “intensity of the pheromone” (the numerical value) increases when more documents are dropped into the cell — and that value decreases if the numbers that represent documents are moved out of the cell. There are several publications in the literature about designing data mining algorithms based the swarm intelligence in ant colonies.